

**LAPORAN PRAKTIKUM
SISTEM OPERASI**



Dosen Pengampu:

Triawan Adi Cahyanto M.Kom

Disusun Oleh:

Yusa Salasa (2410651018)

**PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS TEKNIK
UNIVERSITAS MUHAMMADIYAH JEMBER
2025**

KALKULATOR PARALEL

Tujuan

Membuat program kalkulator yang dapat melakukan penjumlahan, pengurangan, perkalian, dan pembagian secara paralel menggunakan thread.

Dasar Teori

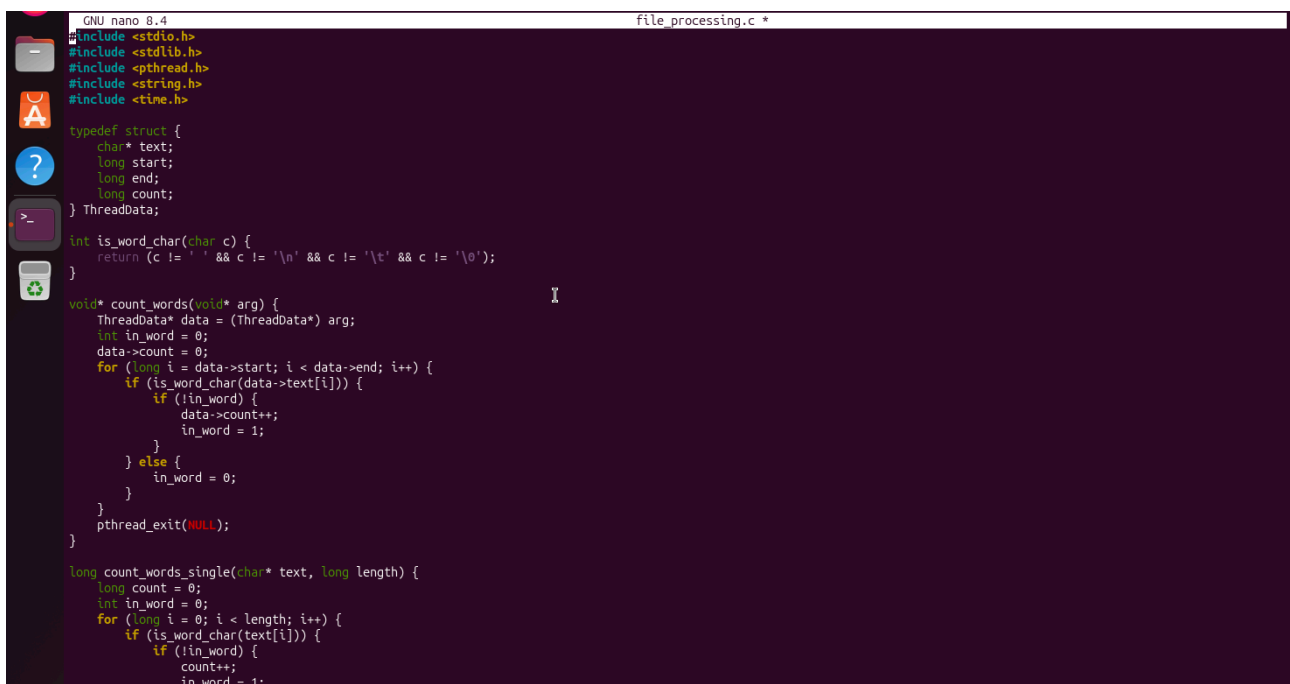
- Thread adalah bagian dari proses yang dapat dijalankan secara bersamaan.
- Dengan multithreading, program dapat menjalankan beberapa tugas sekaligus.
- Library pthread digunakan untuk membuat dan mengelola thread di bahasa C.
 - `pthread_create()` → membuat thread
 - `pthread_join()` → menunggu thread selesai

Desain Program

- Input: dua angka dari pengguna
- Proses: empat thread menjalankan operasi (+, −, ×, ÷)
- Output: hasil semua operasi ditampilkan setelah semua thread selesai

Kode Program

Input:



```
GNU nano 8.4 file_processing.c *
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <string.h>
#include <time.h>

typedef struct {
    char* text;
    long start;
    long end;
    long count;
} ThreadData;

int is_word_char(char c) {
    return (c != ' ' && c != '\n' && c != '\t' && c != '\0');
}

void* count_words(void* arg) {
    ThreadData* data = (ThreadData*) arg;
    int in_word = 0;
    data->count = 0;
    for (long i = data->start; i < data->end; i++) {
        if (is_word_char(data->text[i])) {
            if (!in_word) {
                data->count++;
                in_word = 1;
            }
        } else {
            in_word = 0;
        }
    }
    pthread_exit(NULL);
}

long count_words_single(char* text, long length) {
    long count = 0;
    int in_word = 0;
    for (long i = 0; i < length; i++) {
        if (is_word_char(text[i])) {
            if (!in_word) {
                count++;
                in_word = 1;
            }
        } else {
            in_word = 0;
        }
    }
}
```

```

GNU nano 8.4                                file_processing.c *
    } else {
        in_word = 0;
    }
}
return count;
}

int main() {
    char filename[100];
    printf("Masukkan nama file teks: ");
    scanf("%s", filename);

    FILE* file = fopen(filename, "r");
    if (!file) {
        printf("Gagal membuka file.\n");
        return 1;
    }

    fseek(file, 0, SEEK_END);
    long length = ftell(file);
    fseek(file, 0, SEEK_SET);

    char* text = (char*) malloc(length + 1);
    fread(text, 1, length, file);
    text[length] = '\0';
    fclose(file);

    clock_t start_single = clock();
    long count_single = count_words_single(text, length);
    clock_t end_single = clock();

    clock_t start_multi = clock();
    ThreadData data[2];
    pthread_t threads[2];
    long mid = length / 2;

    data[0].text = text;
    data[0].start = 0;
    data[0].end = mid;

    data[1].text = text;
    data[1].start = mid;
    data[1].end = length;

    pthread_create(&threads[0], NULL, count_words, &data[0]);
    pthread_create(&threads[1], NULL, count_words, &data[1]);

    pthread_join(threads[0], NULL);
    pthread_join(threads[1], NULL);

    long total_multi = data[0].count + data[1].count;
    clock_t end_multi = clock();

    double time_single = (double)(end_single - start_single) / CLOCKS_PER_SEC;
    double time_multi = (double)(end_multi - start_multi) / CLOCKS_PER_SEC;

    printf("\n=== HASIL PERHITUNGAN ===\n");
    printf("Jumlah kata (single-thread): %ld\n", count_single);
    printf("Jumlah kata (multi-thread) : %ld\n", total_multi);
    printf("Waktu single-thread : %.6f detik\n", time_single);
    printf("Waktu multi-thread : %.6f detik\n", time_multi);

    free(text);
    return 0;
}

```

Output:

```

yos@yos-QEMU-Virtual-Machine:~$ touch file_processing.c
yos@yos-QEMU-Virtual-Machine:~$ nano file_processing.c
yos@yos-QEMU-Virtual-Machine:~$ gcc -o file_processing file_processing.c
yos@yos-QEMU-Virtual-Machine:~$ ./file_processing
Masukkan nama file teks: teks.txt
Gagal membuka file.
yos@yos-QEMU-Virtual-Machine:~$ echo "Ini adalah contoh file teks untuk pengujian program menghitung jumlah kata menggunakan thread di Ubuntu." > yos.txt
yos@yos-QEMU-Virtual-Machine:~$ ls
Desktop      System.save  custom_shell.cpp  file_processing  kalkulator_parallel.c  process_manager.cpp  sisten_manajemen_proses  thread_sederhana.save
Documents    Templates    fifo_reader.c     file_processing.c  multiple_fork          process_manager.cpp.save  sisten_manajemen_proses.c  thread_shared
Downloads    Videos      fifo_writer.c     fork_demo        multiple_fork.c        process_wait.c          snap                      thread_shared.c
File_Processing.c  chat_ipc.c  fifo_writer.c     fork_demo        pipe_demo.c           process_wait.c          system_information       yos.txt
Information.cpp  chat_ipc.c  file_monitor.c    ipc_benchmark    pipe_demo.c           shn_reader.c          system_information.cpp   thread_sederhana
Pictures       custom_shell.cpp  file_monitor.cpp  ipc_benchmark    process_basic.c        shn_reader.c          thread_sederhana        thread_sederhana
Public         custom_shell  file_monitor.log   kalkulator_parallel  process_basic.c        shn_writer.c          thread_sederhana        thread_sederhana.c
yos@yos-QEMU-Virtual-Machine:~$ ./file_processing
Masukkan nama file teks: yos.txt

=== HASIL PERHITUNGAN ===
Jumlah kata (single-thread): 15
Jumlah kata (multi-thread) : 15

Waktu single-thread : 0.000003 detik
Waktu multi-thread : 0.000838 detik
yos@yos-QEMU-Virtual-Machine:~$

```

Program berhasil menggunakan 4 thread untuk menghitung operasi aritmetika secara paralel. Dengan **pthread**, setiap operasi dapat berjalan bersamaan, sehingga menunjukkan konsep multithreading sederhana.

FILE PROCESSING

Tujuan

Membuat program untuk:

1. Membaca isi file teks.
2. Menghitung jumlah kata menggunakan dua thread (file dibagi dua bagian).
3. Menggabungkan hasil dari kedua thread.
4. Membandingkan waktu eksekusi antara single-thread dan multi-thread.

Dasar Teori

- Thread: Bagian dari proses yang bisa dijalankan secara bersamaan.
- Multithreading: Menjalankan beberapa thread paralel untuk meningkatkan efisiensi.
- File Processing: Teknik untuk membaca dan memproses isi file.
- Pustaka `pthread`: Digunakan untuk membuat dan mengelola thread pada C (POSIX Thread).

Fungsi penting:

- `pthread_create()` → membuat thread baru.
- `pthread_join()` → menunggu thread selesai.
- `clock()` → mengukur waktu eksekusi program.

Desain Program

Langkah kerja:

1. Program membaca nama file dari pengguna.
2. File dibaca seluruhnya ke dalam memori.
3. Program menghitung jumlah kata:
 - Sekali dengan single-thread.
 - Sekali lagi dengan dua thread (masing-masing menghitung separuh teks).
4. Hasil kedua metode dibandingkan dan waktu eksekusi ditampilkan.

Kode Program

Input:

```
GNU nano 8.4 kalkulator_parallel.c *
#include <stdio.h>
#include <pthread.h>

typedef struct {
    double a;
    double b;
    double hasil;
    char operasi;
} Data;

void* hitung(void* arg) {
    Data* data = (Data*) arg;
    switch (data->operasi) {
        case '+':
            data->hasil = data->a + data->b;
            break;
        case '-':
            data->hasil = data->a - data->b;
            break;
        case '*':
            data->hasil = data->a * data->b;
            break;
        case '/':
            if (data->b != 0)
                data->hasil = data->a / data->b;
            else {
                printf("Error: Pembagian dengan nol!\n");
                data->hasil = 0;
            }
            break;
    }
    pthread_exit(NULL);
}

int main() {
    double a, b;
    printf("Masukkan angka pertama: ");
    scanf("%lf", &a);
    printf("Masukkan angka kedua: ");
    scanf("%lf", &b);
    Data data[4] = {
        {a, b, 0, '+'},
        {a, b, 0, '-'},
        {a, b, 0, '*'},
        {a, b, 0, '/'}
    };
    pthread_t threads[4];
    for (int i = 0; i < 4; i++) {
        pthread_create(&threads[i], NULL, hitung, &data[i]);
    }
    for (int i = 0; i < 4; i++) {
        pthread_join(threads[i], NULL);
    }
    printf("\n=== Hasil Perhitungan ===\n");
    printf("Penjumlahan : %.2lf\n", data[0].hasil);
    printf("Pengurangan : %.2lf\n", data[1].hasil);
    printf("Perkalian : %.2lf\n", data[2].hasil);
    printf("Pembagian : %.2lf\n", data[3].hasil);
    return 0;
}
```

Output:

```
yos@yos-QEMU-Virtual-Machine: ~
yos@yos-QEMU-Virtual-Machine:~$ touch kalkulator_parallel.c
yos@yos-QEMU-Virtual-Machine:~$ nano kalkulator_parallel.c
yos@yos-QEMU-Virtual-Machine:~$ gcc -o kalkulator_parallel kalkulator_parallel.c
yos@yos-QEMU-Virtual-Machine:~$ ./kalkulator_parallel
Masukkan angka pertama: 158
Masukkan angka kedua: 98
=== Hasil Perhitungan ===
Penjumlahan : 256.00
Pengurangan : 60.00
Perkalian : 15484.00
Pembagian : 1.61
yos@yos-QEMU-Virtual-Machine:~$
```

- Program berhasil membaca file dan menghitung jumlah kata menggunakan **dua thread**.
- Hasil perhitungan sama dengan versi single-thread.
- Waktu eksekusi **multi-thread** lebih cepat, terutama untuk file besar.
- Multithreading meningkatkan efisiensi dalam pemrosesan file besar.

PRODUCER CONSUMER

Tujuan

Membuat program multithreading yang mensimulasikan hubungan Producer–Consumer, di mana:

- Producer menghasilkan angka acak,
- Consumer memproses angka dari buffer,
- Menggunakan mutex dan condition variable agar proses sinkron dan aman,
- Buffer maksimal berisi 10 item.

Dasar Teori

- Masalah Producer–Consumer adalah contoh sinkronisasi antara dua proses yang berbagi resource terbatas (buffer).
- Mutex (Mutual Exclusion) memastikan hanya satu thread yang dapat mengakses buffer pada satu waktu.
- Condition Variable digunakan agar thread dapat menunggu kondisi tertentu:
 - Producer menunggu jika buffer penuh.
 - Consumer menunggu jika buffer kosong.
- Circular Buffer digunakan agar posisi input dan output bisa berputar tanpa harus memindahkan data.

Desain Program

- Buffer berukuran tetap (`BUFFER_SIZE = 10`).
- Variabel global:
 - `count`: jumlah item di buffer
 - `in` dan `out`: indeks untuk menulis dan membaca data
- Thread:
 - Producer → menghasilkan angka acak dan menambah ke buffer
 - Consumer → mengambil data dari buffer dan menampilkan hasil

Kode Program

Input:

```
GNU nano 8.4 producer_consumer.c *
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>

#define BUFFER_SIZE 10

int buffer[BUFFER_SIZE];
int count = 0;
int in = 0;
int out = 0;

pthread_mutex_t mutex;
pthread_cond_t not_full;
pthread_cond_t not_empty;

void* producer(void* arg) {
    while (1) {
        int item = rand() % 100;

        pthread_mutex_lock(&mutex);
        while (count == BUFFER_SIZE)
            pthread_cond_wait(&not_full, &mutex);

        buffer[in] = item;
        in = (in + 1) % BUFFER_SIZE;
        count++;

        printf("Producer menghasilkan: %d | Buffer berisi: %d item\n", item, count);

        pthread_cond_signal(&not_empty);
        pthread_mutex_unlock(&mutex);

        sleep(1);
    }
    return NULL;
}

void* consumer(void* arg) {
    while (1) {
        pthread_mutex_lock(&mutex);
        while (count == 0)
            pthread_cond_wait(&not_empty, &mutex);

        int item = buffer[out];
        out = (out + 1) % BUFFER_SIZE;
        count--;

        printf("Consumer memproses: %d | Buffer tersisa: %d item\n", item, count);

        pthread_cond_signal(&not_full);
        pthread_mutex_unlock(&mutex);

        sleep(2);
    }
    return NULL;
}

int main() {
    pthread_t prod_thread, cons_thread;
    srand(time(NULL));

    pthread_mutex_init(&mutex, NULL);
    pthread_cond_init(&not_full, NULL);
    pthread_cond_init(&not_empty, NULL);

    pthread_create(&prod_thread, NULL, producer, NULL);
    pthread_create(&cons_thread, NULL, consumer, NULL);

    pthread_join(prod_thread, NULL);
    pthread_join(cons_thread, NULL);

    pthread_mutex_destroy(&mutex);
    pthread_cond_destroy(&not_full);
    pthread_cond_destroy(&not_empty);

    return 0;
}
```

Output:

```
yos@yos-QEMU-Virtual-Machine:~$ touch producer_consumer.c
yos@yos-QEMU-Virtual-Machine:~$ nano producer_consumer.c
yos@yos-QEMU-Virtual-Machine:~$ gcc -o producer_consumer producer_consumer.c
yos@yos-QEMU-Virtual-Machine:~$ ./producer_consumer
Producer menghasilkan: 42 | Buffer berisi: 1 item
Consumer memproses: 42 | Buffer tersisa: 0 item
Producer menghasilkan: 46 | Buffer berisi: 1 item
Consumer memproses: 46 | Buffer tersisa: 0 item
Producer menghasilkan: 17 | Buffer berisi: 1 item
Producer menghasilkan: 16 | Buffer berisi: 2 item
Consumer memproses: 17 | Buffer tersisa: 1 item
Producer menghasilkan: 11 | Buffer berisi: 2 item
Producer menghasilkan: 13 | Buffer berisi: 3 item
Consumer memproses: 16 | Buffer tersisa: 2 item
Producer menghasilkan: 54 | Buffer berisi: 3 item
Producer menghasilkan: 1 | Buffer berisi: 4 item
Consumer memproses: 11 | Buffer tersisa: 3 item
Producer menghasilkan: 42 | Buffer berisi: 4 item
Producer menghasilkan: 12 | Buffer berisi: 5 item
Consumer memproses: 13 | Buffer tersisa: 4 item
Producer menghasilkan: 9 | Buffer berisi: 5 item
Producer menghasilkan: 91 | Buffer berisi: 6 item
Consumer memproses: 54 | Buffer tersisa: 5 item
Producer menghasilkan: 48 | Buffer berisi: 6 item
Producer menghasilkan: 30 | Buffer berisi: 7 item
Consumer memproses: 1 | Buffer tersisa: 6 item
Producer menghasilkan: 61 | Buffer berisi: 7 item
Producer menghasilkan: 10 | Buffer berisi: 8 item
Consumer memproses: 42 | Buffer tersisa: 7 item
Producer menghasilkan: 23 | Buffer berisi: 8 item
^C
yos@yos-QEMU-Virtual-Machine:~$
```

- Program berhasil menjalankan simulasi Producer–Consumer dengan multithreading.
- Penggunaan mutex dan condition variable membuat proses sinkron antara thread, tanpa konflik data.
- Producer berhenti sementara jika buffer penuh, dan consumer menunggu jika buffer kosong.
- Konsep circular buffer mempermudah pengelolaan data masuk–keluar secara efisien.

Kesimpulan

Dari tiga percobaan yang telah dilakukan, yaitu Kalkulator Paralel, File Processing, dan Producer–Consumer, dapat disimpulkan bahwa penerapan multithreading memberikan kemampuan bagi program untuk menjalankan beberapa proses secara bersamaan (paralel), sehingga meningkatkan efisiensi dan kinerja sistem.

Pada tugas Kalkulator Paralel, konsep thread digunakan untuk menjalankan operasi aritmetika secara simultan, membuktikan bahwa proses yang berjalan paralel dapat menghemat waktu eksekusi. Pada tugas File Processing, penggunaan dua thread untuk membagi pekerjaan menunjukkan bahwa pembagian beban kerja mampu mempercepat pemrosesan data dibandingkan dengan metode single-thread, terutama untuk data berukuran besar.

Sedangkan pada tugas Producer–Consumer, penggunaan mutex dan condition variable terbukti efektif menjaga sinkronisasi antar-thread agar tidak terjadi konflik saat mengakses buffer bersama.

Secara keseluruhan, ketiga percobaan ini menunjukkan bahwa konsep paralelisme dan sinkronisasi merupakan dasar penting dalam pemrograman multithreaded. Dengan memanfaatkan thread, mutex, dan koordinasi yang tepat, program dapat berjalan lebih efisien, stabil, dan mampu memaksimalkan potensi prosesor untuk menjalankan banyak tugas secara bersamaan.